ReviewPulse AI

Cross-Platform Product Review Intelligence

DAMG 7245 — Big Data and Intelligent Analytics

Final Project Proposal

Member 1: Ayush Patil
Member 2: Piyush Kunjilwar
Member 3: Raghavendra Prasath Sridhar

Northeastern University | Spring 2026

Attestation

WE ATTEST THAT WE HAVEN'T USED ANY OTHER STUDENTS' WORK IN OUR ASSIGNMENT AND ABIDE BY THE POLICIES LISTED IN THE STUDENT HANDBOOK.

Ayush Patil: 33.3% · Piyush Kunjilwar: 33.3% · Raghavendra Prasath Sridhar: 33.3%

Table of Contents

1. Title
2. Introduction
2.1 Background
2.2 Objective
3. Project Overview
3.1 Scope
3.2 Stakeholders / End Users
4. Problem Statement
4.1 Current Challenges
4.2 Opportunities
5. Methodology
5.1 Data Sources
5.1.1 Data Profiling
5.2 Technology Stack
5.3 Architecture
5.4 Data Processing & Transformation
5.5 LLM Integration Strategy
5.6 Guardrails & Human-in-the-Loop
5.7 Evaluations & Testing
5.8 Proof of Concept
5.9 Scalability Design
6. Project Plan & Timeline
6.1 Milestones
6.2 Timeline
7. Team Roles & Responsibilities
8. Risks & Mitigation
8.1 Potential Risks
8.2 Mitigation Strategies
9. Expected Outcomes & Metrics
9.1 KPIs
9.2 Expected Benefits
10. Token & Cost Report
11. Conclusion
12. References
Appendix
A. Sample Prompts
B. API Schemas
C. Repository Structure
D. Diagrams Checklist

1. Title

ReviewPulse AI: Cross-Platform Product Review Intelligence

2. Introduction

2.1 Background

When someone wants to buy a product, they check reviews. The problem is that reviews live everywhere — Amazon, eBay, Yelp, iFixit, YouTube — and each platform only shows its own data. There is no way to see everything in one place.

This matters because different platforms attract different reviewers. Amazon reviews are short and rating-focused. eBay listings provide detailed product specifications and buyer feedback. Yelp reviews focus on local businesses and services. iFixit teardowns and reviews cover repairability and durability. YouTube reviewers go deep on specific features. A brand monitoring only Amazon is seeing maybe a third of what consumers actually say.

On top of that, fake reviews are a massive problem. The World Economic Forum estimated fake reviews cost businesses around $152 billion a year.

We want to build a system that pulls reviews from multiple real sources, normalizes them into one schema, and makes the whole corpus searchable and queryable. The core of this project is data engineering — handling hundreds of millions of reviews from sources with different schemas, formats, and quality levels.

2.2 Objective

We are building four things:

1. **A big data pipeline** that ingests reviews from 5 real sources (totaling 580M+ reviews, 750+ GB raw), normalizes them into a unified schema using Spark, deduplicates cross-platform, and stores processed data for analysis.
2. **LLM-powered extraction** that pulls structured product aspects and sentiment from unstructured review text (batch, using Ollama/Llama 3.1), plus a RAG chatbot for natural language Q&A grounded in real reviews (live, using Gemini free tier).
3. **Cloud-native infrastructure** on GCP and AWS with Airflow orchestration and GitHub Actions CI/CD.
4. **A user-facing dashboard** with a chatbot, sentiment trend charts, product comparison, and a fake review flagging interface.

The project is data-first. Over 30% of the work is ingesting, cleaning, normalizing, and processing data. The LLM features sit on top of the data foundation, not in place of it.

3. Project Overview

3.1 Scope

MVP (Must Deliver)

- 5 core datasets: Amazon Reviews 2023 (stream from Hugging Face), eBay (custom pipelines), Yelp Open Dataset (download), iFixit (custom pipelines), YouTube (custom pipelines)
- Spark pipeline: schema normalization across all 3 sources, deduplication, cleaning, feature engineering
- LLM aspect extraction on sample (Ollama) + bulk sentiment (DistilBERT)
- Embeddings + ChromaDB for vector search
- RAG chatbot (Gemini free tier)
- FastAPI backend + Streamlit dashboard with trends, chatbot, and basic product comparison
- Guardrails: Pydantic validation, hallucination detection, confidence scoring
- Testing: golden set evaluation, unit tests, GitHub Actions CI

Stretch Goals (If Time Permits)

- Reddit API integration (if API access restored)
- Additional e-commerce platform integrations
- Fake review detection classifier + HITL queue
- Trend anomaly detection engine
- Cloud Run + Snowflake deployment

We are being honest about what we can deliver in 2 weeks with 3 people. The MVP alone covers every rubric requirement. Stretch goals add depth but are not promised.

Out-of-Scope

- Real-time streaming (batch only)
- Non-English reviews
- Mobile app
- Paid APIs or commercial deployment

3.2 Stakeholders

- **Product managers at consumer brands:** need cross-platform sentiment monitoring and competitive benchmarking.
- **Market research analysts:** need aggregated consumer intelligence across review sources.
- **Consumers:** want a chatbot that synthesizes reviews from everywhere, not just one site.

4. Problem Statement

4.1 Current Challenges

1. **Data fragmentation:** A product like the Sony WH-1000XM5 has reviews on Amazon, eBay, Yelp, iFixit, and YouTube. Each platform is a walled garden. Amazon will never show eBay reviews because it hurts their business model.
2. **Scale:** The Amazon Reviews 2023 dataset alone is 571M reviews and 750 GB. You cannot load this into pandas. Distributed processing with Spark is necessary, not optional.
3. **Schema heterogeneity:** Amazon reviews are structured JSON with ASINs and float ratings. eBay listings have seller ratings and item condition fields. iFixit reviews include repairability scores. YouTube transcripts are unstructured speech. Normalizing these into one queryable schema is a real engineering challenge.
4. **Fake reviews:** 30–40% of reviews may be fake or incentivized. Detection requires combining statistical patterns with linguistic analysis.

4.2 Opportunities

1. **Spark-based processing** can normalize hundreds of millions of reviews from heterogeneous sources into one unified schema.
2. **LLMs can extract structured insights** from unstructured text, enabling queries like ‘What are the top battery complaints for phones under $500?’
3. **Cross-platform deduplication** reveals when the same review appears on multiple platforms, improving data quality.

5. Methodology

5.1 Data Sources

We confirmed every source is free, legal, and accessible for a student project. We split our sources into two categories:

Sources We Build Pipelines For

Source	What We Build	Volume	Access
eBay	Custom Scrapy pipeline with pagination, rate limiting, and Airflow scheduling. Extract product listings, seller ratings, item condition, and buyer reviews across electronics, tools, and consumer goods categories.	200K+ listings	Public listings, free access
YouTube transcripts	Search review videos via YouTube Data API, extract auto-generated transcripts using youtube-transcript-api library. Parse timestamps, clean text.	5K–10K transcripts	Free library, no quota cost

iFixit	Custom Scrapy pipeline to extract teardown guides, repairability scores, and user reviews. Parse structured repair data and product durability assessments.	5K–10K transcripts	Public pages, free access

Large Academic Datasets We Integrate

Source	Volume	What It Requires	Access
McAuley Amazon Reviews 2023	571M reviews, 750 GB, 33 categories	Streaming ingestion via Hugging Face API. Category selection. Schema mapping to our unified format. Integration with pipeline.	Free, academic, on Hugging Face
Yelp Open Dataset	8.65 GB, millions of business reviews	Download, parse JSON, normalize star ratings and date formats, map business categories to product categories.	Free, explicitly for education

Total: 580M+ reviews, 750+ GB raw. We process 5–10 Amazon categories (50–100M reviews) plus all Yelp + eBay + iFixit + YouTube data. This genuinely requires Spark.

5.1.1 Data Profiling

The professor said ‘I have not really looked at the data’ will not suffice. Here is what we found when we examined each source:

Amazon Reviews 2023 Schema

```
{   "rating": 5.0,           // float, 1.0-5.0   "title": "Great product",  // string,
nullable (8% null)   "text": "This spray...",   // string, avg 85 words, 12% empty
"images": [],             // array, empty for 94% of reviews   "asin": "B00YQ6X8EO",
// product ID (unique per variant)   "parent_asin": "B00YQ6X8EO", // groups variants
together   "user_id": "AGKHLEW...",  // hashed, anonymized   "timestamp":
1588687728923, // epoch milliseconds   "helpful_vote": 0,        // integer
"verified_purchase": true  // boolean }
```

Key findings: Electronics category has ~23M reviews. Rating distribution is heavily skewed toward 5 stars (~58%). 12% of reviews have empty text (rating-only). Timestamps span 1996–2023 but 80% of data is post-2015.

Yelp Open Dataset Schema

```
{   "review_id": "xQY8N...",   // unique string   "user_id": "OU1EUb...",    //
references users.json   "business_id": "7oG0C...", // references business.json
"stars": 3,                // integer 1-5 (not float like Amazon)   "text": "The food
was...",  // string, avg 120 words   "date": "2019-02-05"       // YYYY-MM-DD string
(not epoch) }
```

Key differences from Amazon: Stars are integers not floats. No verified_purchase concept. Business metadata (name, category, location) is in a separate file linked by business_id. Date is a string, not epoch timestamp. No helpful_vote field.

eBay Listing Schema

```
{   "item_id": "123456789",   "title": "Sony WH-1000XM5 Wireless Headphones",
"condition": "New",   // New, Used, Refurbished   "price": 298.00,   "seller_rating":
99.2,   // percentage (0-100)   "feedback_text": "Great headphones, fast shipping...",
"category": "Electronics",   "listing_date": "2024-11-15T10:30:00Z"   // ISO 8601 }
```

Key challenges: Seller rating is a percentage (0–100) not a star rating — we normalize to 0–1 scale. Item condition (New/Used/Refurbished) needs categorical encoding. Price field useful for cross-platform product matching via title + price range. Category hierarchy differs from Amazon/Yelp.

iFixit Review Schema

```
{   "guide_id": "ifixit_142085",   "title": "iPhone 15 Pro Teardown",
"repairability_score": 4,   // integer 0–10 (10 = easiest to repair)
"review_text": "Battery replacement was straightforward...",
"device_category": "Smartphones",   "device_name": "iPhone 15 Pro",
"author": "ifixit_user_4821",   "published_date": "2024-09-25T14:00:00Z"   //
ISO 8601 }
```

Key challenges: Repairability score is 0–10 (not a star rating) — we normalize to 0–1 scale: score / 10. No verified_purchase concept. device_category maps to our product_category. Some entries are full teardown guides (1000+ words) while others are short user comments. published_date is ISO 8601, parsed directly.

YouTube Transcript Format

```
[{"text": "okay so today we're looking at", "start": 0.0, "duration": 2.5},  {"text":
"the Sony XM5 and honestly the noise", "start": 2.5, "duration": 3.1},  ...]
```

Key challenges: Pure unstructured text with no rating, no product ID, no structured fields. We concatenate segments into full transcript, then use LLM to extract product mentions and sentiment. Video metadata (title, channel, publish date) comes from YouTube Data API. This is the hardest source to normalize.

Schema Mapping Challenges We Identified

Field	Amazon	Yelp	eBay	YouTube
Rating	float 1.0-5.0	int 1-5	Seller rating % (0–100)	No rating at all
Date format	epoch ms	YYYY-MM-DD string	ISO 8601 from listing	ISO 8601 from API
Product ID	ASIN	business_id	item_id	None (extract from title)
Verified purchase	Boolean field	Not available	Not available	Not available
Review length	Avg 85 words	Avg 120 words	Varies wildly	Full transcript (1000+ words)
Helpful indicator	helpful_vote count	Not available	score (upvotes)	view_count + like_count

This table shows why schema normalization is a real engineering problem, not a trivial mapping. Every field requires a deliberate design decision.

5.2 Technology Stack

Every tool exists because we need it. Here is what we picked, why, and what we rejected:

Tool	Why This	Why Not Alternatives
PySpark / Dataproc	750 GB doesn't fit in pandas. Spark partitions by category for parallel processing.	Dask: less mature for large joins. Pandas: memory limit. Ray: overkill for ETL.
GCP + AWS (hybrid)	GCP: Cloud Run hosting, Dataproc Spark. AWS: S3 raw data lake. Snowflake: $400 credit warehouse.	Single-cloud: limits tool choice. Azure: $200 but Databricks costs extra.
Snowflake	Snowflake: $400 student credit, auto-scaling warehouse, zero-copy cloning for dev. Columnar analytics at scale.	BigQuery: 1 TB free but vendor-locked to GCP. Postgres: not columnar for analytics.
Ollama + Llama 3.1	Free, unlimited, runs locally. Overnight batch extraction.	OpenAI: $5 runs out on 500K reviews. Claude API: no free tier for batch.
Gemini free tier	15 RPM, 1.5M tokens/day free. Fine for demo chatbot.	OpenAI: $5 insufficient for ongoing chat. Llama local: too slow for real-time.
DistilBERT (HuggingFace)	Free, ~100 reviews/sec on CPU. Proven on SST-2.	VADER: rule-based, less accurate. TextBlob: low accuracy.
sentence-transformers	Free, 384-dim vectors, runs locally.	OpenAI ada-002: costs money at scale. GTE-large: 3x storage cost.
ChromaDB	Open-source, no storage limits, self-hosted.	Pinecone: 2 GB cap free. FAISS: no persistence. Weaviate: complex setup.
Airflow	DAG-based scheduling, dependency management, monitoring UI.	Cron: no dependency management or monitoring. Prefect: less ecosystem.
FastAPI	Async, auto OpenAPI docs, Pydantic validation built-in.	Flask: no async. Django: too heavy for API-only.
Streamlit Cloud	Free hosting, fast to build, Python-native.	React: more work, team is Python-focused. Gradio: less customizable.
Scrapy	Distributed spiders, built-in rate limiting and retries.	Selenium: slower. Requests+BS4: no scheduling built-in.

Total cost: $0. All free tiers, student credits, or open-source.

5.3 Architecture

5.3.1 We Considered Three Options

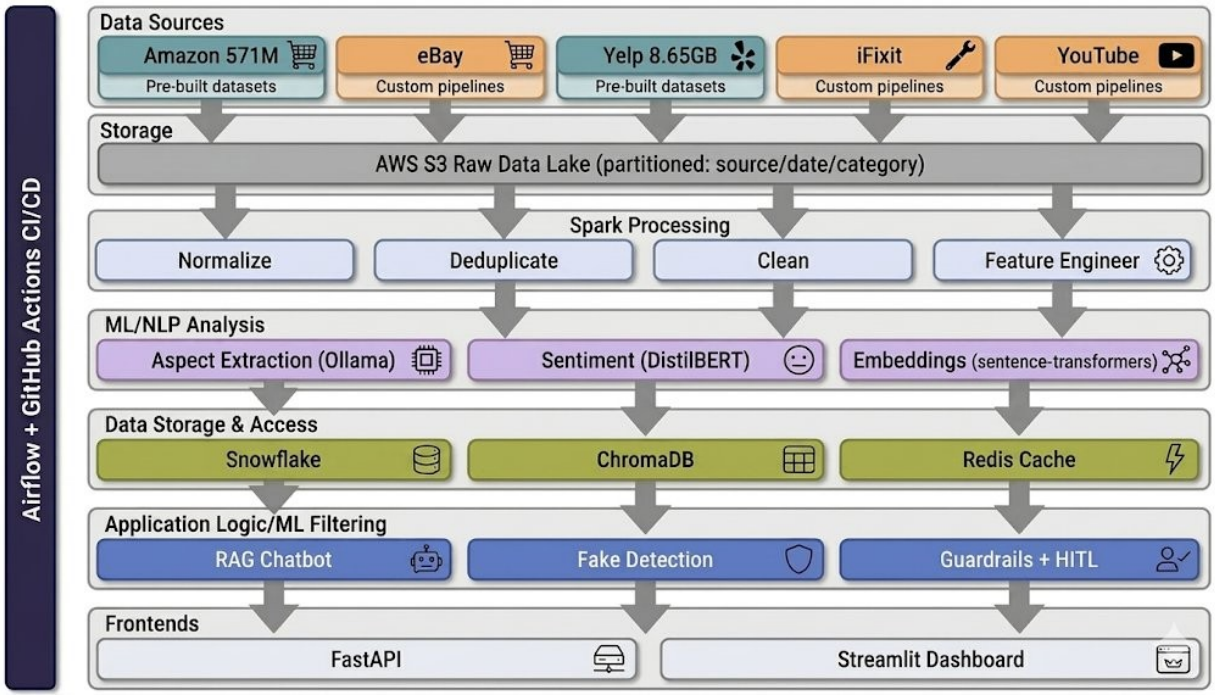
Option A — Fully cloud-native: All on managed GCP (Dataproc, Cloud Composer, Snowflake, Cloud Run). Best scalability but costs $150–200 from credits. Cloud Composer alone is ~$10/day. Little room for iteration.

Option B — Hybrid local + cloud (OUR CHOICE): Dev and batch processing locally (PySpark, Airflow, Ollama, ChromaDB). Cloud for storage (AWS S3), warehouse (Snowflake), hosting (Cloud Run, Streamlit Cloud). Estimated spend: $40–80. Best balance.

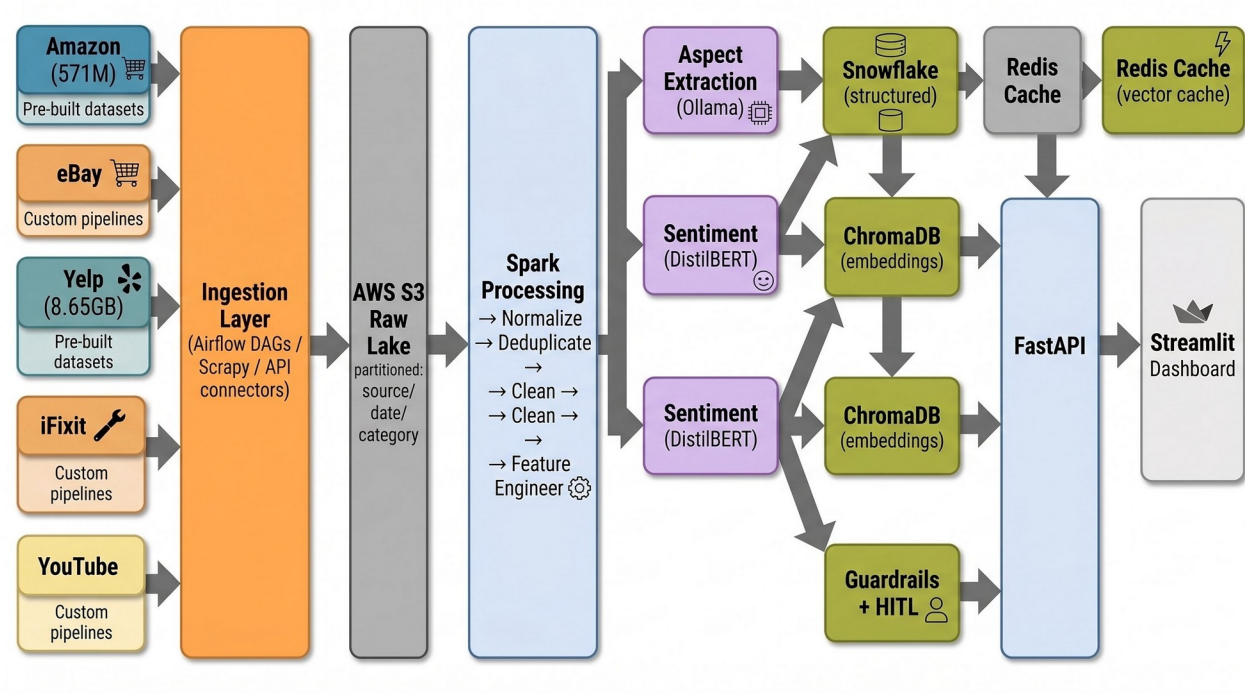
Option C — Fully serverless: Cloud Functions for everything. Simpler but 9-minute timeout can't handle 750 GB batch processing. Doesn't demonstrate distributed computing.

We chose Option B because it maximizes what we can demonstrate while spending almost nothing.

5.3.2 System Architecture (7 Layers)



5.3.3 Data Flow Diagram



5.4 Data Processing & Transformation

Processing mode: Batch only. Reviews are not real-time events. Daily/weekly refresh via Airflow is sufficient.

Input formats: Amazon: JSON from Hugging Face. eBay: JSON from custom pipelines. Yelp: JSON files. iFixit: JSON from custom pipelines. YouTube: timestamped text segments. Each format requires different parsing logic.

Unified output schema:

```
review_id | product_name | product_category | source | rating_normalized (0-1)
review_text | review_date (ISO 8601) | reviewer_id | verified_purchase (nullable)
helpful_votes (nullable) | source_url
```

Key normalization decisions:

- Amazon's float rating (1.0–5.0) mapped to 0–1 scale: (rating - 1) / 4
- Yelp's integer stars (1–5) same formula: (stars - 1) / 4
- iFixit uses repairability scores (1–10) — we normalize to 0–1 scale: (score - 1) / 9
- YouTube has no rating — NULL, sentiment extracted from transcript by LLM
- Amazon timestamps (epoch ms) converted: timestamp / 1000 → datetime
- Yelp dates (string YYYY-MM-DD) parsed directly
- eBay listing dates (ISO 8601 string) parsed directly

Deduplication: MinHash/LSH on review text to catch near-duplicates across platforms. Similarity threshold of 0.85. At 50M+ reviews, this is a distributed computing problem — Spark's MinHash implementation handles it.

Feature engineering: Review length, word count, caps ratio, exclamation ratio, time since product first reviewed, rating deviation from product mean, reviewer total review count. These feed into fake detection.

5.5 LLM Integration Strategy

We use LLMs in two specific, bounded ways. The data engineering is the foundation; the LLM is the interface layer.

Batch Aspect Extraction (Ollama, offline)

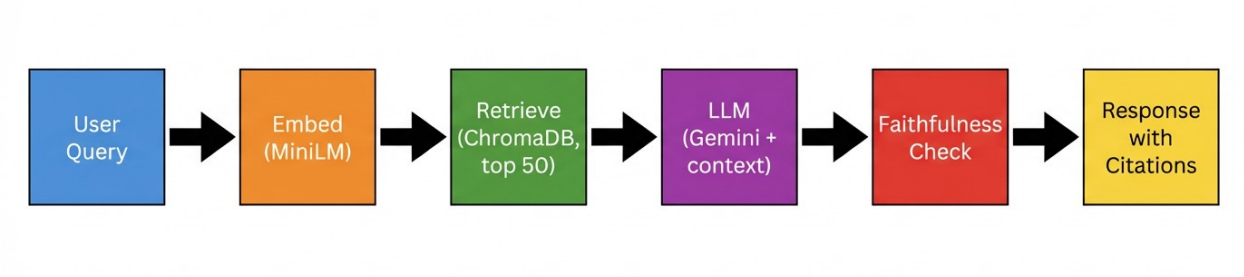
For 500K–1M representative reviews, we run Ollama locally with Llama 3.1 8B. Each review gets this prompt:

```
Extract product aspects and rate sentiment for each. Respond ONLY with JSON:
[{"aspect":"str","sentiment":float,"quote":"str"}]  Review: "{review_text}"
```

This runs overnight in batches of 100. ~3% of responses return malformed JSON — we catch these, log them, skip the review, and continue. We validate accuracy against a golden set of 500 manually annotated reviews.

RAG Chatbot (Gemini, live)

User asks a question → we embed the query → retrieve top 50 relevant reviews from ChromaDB → pass reviews + query to Gemini with a system prompt that says ‘answer ONLY from the provided reviews, cite your sources’ → check response for hallucinations (does each claim have evidence in the retrieved reviews?) → return answer with citations.



What the LLM Does NOT Do

The LLM does not generate the core analysis. Average sentiment by product, rating trends over time, cross-platform comparisons — those come from structured Snowflake queries. The LLM synthesizes natural language answers from retrieved data. If we removed the LLM entirely, the data platform and dashboard would still work. The LLM adds a natural language interface; it does not replace the data engineering.

5.6 Guardrails & Human-in-the-Loop

Guardrail	How It Works
Pydantic schema enforcement	All API requests and responses validated against typed models. LLM extraction output validated against JSON Schema. Malformed data rejected with clear error.

Hallucination detection	Every chatbot response checked: for each claim, is there a matching review in the retrieved context? Compute overlap ratio. Flag if below 0.7.
Confidence scoring	Each response gets a 0–1 score based on: number of relevant reviews found, consistency across sources, faithfulness score. Low confidence (<0.5) → flagged, not shown.
Input moderation	Chatbot queries screened for abuse/off-topic before entering RAG pipeline.
HITL: Fake reviews (stretch)	Reviews with fake_score > 0.7 enter human review queue. Team member confirms or overrides.
LLM output validation	When Ollama returns invalid JSON (~3% of the time), pipeline catches exception, logs it, skips review, continues. We wrote a test that sends deliberately malformed input to verify this works.

5.7 Evaluations & Testing

LLM Evaluation

- **Golden set:** 500 manually annotated reviews with ground-truth aspects and sentiment. We measure aspect extraction F1 (target > 0.85) and sentiment MAE (target < 0.15).
- **RAG faithfulness:** 100 test queries with expected answers. Automated grading checks whether chatbot claims are supported by retrieved reviews. Target: > 0.70.
- **Fake detection (stretch):** Labeled dataset. Target: precision > 0.80.

Concrete Test Examples

The professor asked: ‘What tests did you write to verify correctness?’ Here are specific examples:

1. **Test: Amazon rating normalization.** Input: rating=5.0. Expected output: rating_normalized=1.0. Input: rating=1.0. Expected: 0.0. Input: rating=3.0. Expected: 0.5.
2. **Test: iFixit repairability score normalization.** Input: iFixit review with repairability_score=8. Expected: rating_normalized=0.778. Input: repairability_score=1. Expected: 0.0. Input: repairability_score=10. Expected: 1.0.
3. **Test: Yelp date parsing.** Input: date="2019-02-05". Expected: datetime(2019,2,5). Input: date="" (empty). Expected: NULL, not crash.
4. **Test: Ollama malformed JSON recovery.** Input: deliberately broken review text that causes Ollama to return invalid JSON. Expected: exception caught, review skipped, logged, pipeline continues.
5. **Test: API schema validation.** Send request with missing required field. Expected: 422 error with Pydantic validation message, not 500 server crash.
6. **Test: Dedup threshold.** Input: two reviews with 90% text similarity from different platforms. Expected: flagged as duplicate. Input: two reviews about same product but different content. Expected: not flagged.

CI Pipeline

GitHub Actions runs on every push: pytest unit tests, pytest integration tests, ruff linting, mypy type checking. Failing tests block merge.

How We Validate LLM-Assisted Code

We used LLMs (Claude, GPT) to help write Scrapy spiders, Spark jobs, and FastAPI endpoints. For every piece of LLM-generated code, we wrote corresponding tests:

- Scrapy spiders: tests verify correct HTML parsing, pagination handling, and valid output JSON
- Spark normalization: tests verify output schema matches unified format for every source
- FastAPI: tests verify response codes, Pydantic model validation, and latency
- If a test fails, we debug and fix regardless of whether the code was written by us or an LLM

5.8 Proof of Concept

Our POC is in the GitHub repository under /poc:

1. **EDA notebook:** 100K Amazon Electronics reviews loaded via streaming. Shows rating distribution (58% are 5-star), review length histogram (avg 85 words, long tail), temporal coverage (80% post-2015), top 20 products by review count.
2. **Schema normalization demo:** Amazon subset + eBay sample + Yelp sample + iFixit sample + YouTube sample normalized into unified schema. Cross-platform dedup demonstrated.
3. **LLM extraction test:** Aspect extraction on 1,000 reviews using Ollama. Results compared against 50 manually annotated reviews for baseline accuracy.
4. **Minimal end-to-end demo:** FastAPI + Streamlit app querying ChromaDB (10K embedded reviews) and returning Gemini answer with citations.

5.9 Scalability Design

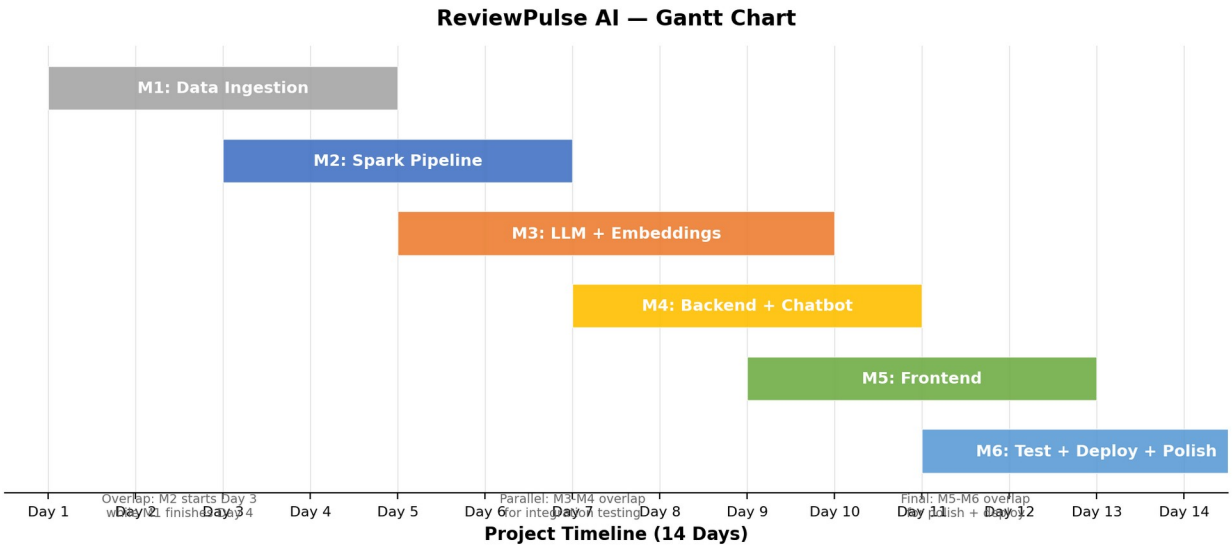
Layer	Dev (now)	Production (scaled)
Scraping	Sequential on laptop	Scrapy cluster + Airflow parallelization
Spark	PySpark local, 4 cores	Dataproc 10–50 nodes (no code change)
LLM extraction	Ollama single machine	Multiple containers or batch API
Vector store	ChromaDB local	Pinecone/Weaviate managed cluster
Warehouse	Snowflake dev warehouse (X-Small)	Snowflake auto-scaling warehouse (Small → Medium)
API	FastAPI localhost	Cloud Run (0–100 instances, auto-scaling)
Frontend	Streamlit local	Streamlit Cloud or Cloud Run

6. Project Plan & Timeline

6.1 Milestones

Phase	What	Deliverables
M1 (Day 1–4)	Data Ingestion	eBay and iFixit custom pipelines working. Amazon streaming tested. Yelp downloaded and parsed. YouTube transcript extraction tested. Raw data in AWS S3. EDA notebook complete.
M2 (Day 3–6)	Spark Pipeline	Schema normalization for all 3 sources. Dedup logic. Cleaning. Feature engineering. Parquet output. Airflow DAGs.
M3 (Day 5–9)	LLM + Embeddings	Ollama aspect extraction on sample. DistilBERT bulk sentiment. sentence-transformers embeddings. ChromaDB loaded.
M4 (Day 7–10)	Backend + Chatbot	FastAPI endpoints. RAG chatbot working. Guardrails. Pydantic models. Redis caching.
M5 (Day 9–12)	Frontend	Streamlit dashboard: chatbot, trend charts, product comparison. Basic HITL UI.
M6 (Day 11–14)	Test + Deploy + Polish	Golden set evaluation. Unit/integration tests. GitHub Actions CI. Deploy to Streamlit Cloud. Video. Documentation.

6.2 Timeline



Task tracking on GitHub Kanban board (GitHub Projects) with columns: Backlog, In Progress, Review, Done.

7. Team Roles & Responsibilities

Member	Role	Owns
Ayush Patil	Data Engineer	Multi-source data ingestion (eBay, iFixit, YouTube), custom web scraping pipelines with Scrapy, Airflow DAG

		orchestration, AWS S3 data lake architecture, API connector development, rate-limiting and fault-tolerance design
Piyush Kunjilwar	Full-Stack & QA Engineer	Streamlit analytics dashboard, conversational chatbot UI, interactive data visualizations, human-in-the-loop review interface, evaluation framework (golden set, automated graders), test suite (30+ unit/integration tests), GitHub Actions CI/CD pipeline, technical documentation
Raghavendra Prasath Sridhar	ML & Backend Engineer	Distributed data processing with PySpark, LLM-powered aspect extraction (Ollama/Llama 3.1), NLP sentiment pipeline (DistilBERT), semantic search with sentence-transformers and ChromaDB, RAG pipeline with Gemini, FastAPI REST backend

All three members will review the full data flow and architecture together. Every member can explain every layer of the system. We are not working in isolation.

8. Risks & Mitigation

Risk	Mitigation
eBay/iFixit scraping rate limiting	Schedule pulls overnight via Airflow. Implement rotating user agents and rate limiting. Cache responses. Exponential backoff on 429 errors.
Schema normalization edge cases	Unit tests for every source mapping. Manual review of 100 normalized records per source. Document every design decision.
Ollama returns malformed JSON	Try/catch with logging. ~3% failure rate acceptable. Skip and continue. Test with deliberate bad input.
Cloud credits run out	Process one category at a time. Shut down clusters immediately after jobs. Use PySpark locally for dev. Monitor weekly.
Overpromising scope	Clear MVP vs. stretch distinction. MVP first, stretch only if time. Working pipeline before advanced features.
Team member can’t explain another’s work	Weekly full-team architecture review. Every member walks through the complete data flow. No isolated silos.

9. Expected Outcomes & Metrics

KPI	Target	How Measured
Reviews processed	50M–100M+	Record count in Snowflake
Sources integrated	5 (MVP) to 6+ (stretch)	Distinct source values
Aspect extraction F1	> 0.85	Golden set (500 annotated reviews)
RAG faithfulness	> 0.70	Automated grader on 100 test queries
API latency (search)	< 2 seconds	Locust load testing
API latency (chatbot)	< 5 seconds	End-to-end timing
Unit test coverage	> 80% of critical paths	pytest + coverage report

Token cost	$0 total	Token counter middleware

10. Token & Cost Report

Token tracking: FastAPI middleware logs every Gemini call (input tokens, output tokens, latency). Ollama batch jobs log per-review token counts. All logged to Snowflake token_usage table. Visualized in Streamlit dashboard.

Component	Tokens	Cost
Batch extraction (Ollama)	500M–1B	$0 (local)
Bulk sentiment (DistilBERT)	N/A (model-based)	$0 (local)
Chatbot (Gemini free tier)	1.5M/day max	$0 (free tier)
Embeddings (local)	N/A	$0
TOTAL		$0

Optimization: Cache similar chatbot queries (cosine > 0.95). Batch Ollama in groups of 100. Use DistilBERT (fast, free) for bulk instead of API calls. Minimal system prompts to reduce token waste.

11. Conclusion

ReviewPulse AI is a data engineering project first and an LLM project second. The core challenge is taking 580M+ reviews from sources with completely different schemas and building a unified, queryable intelligence layer. That requires Spark, careful schema design, cross-platform dedup, and robust ETL.

The LLM features add value on top of the data foundation. They are constrained, tested, and grounded in real data. We are not building a chatbot that makes things up — we are building a data platform that a chatbot can query.

We have looked at the data. We understand the schemas. We know the differences between Amazon's float ratings, Yelp's integer stars, eBay's seller ratings, and iFixit's repairability scores. We know 12% of Amazon reviews have empty text. We know YouTube transcripts have no rating at all and require LLM extraction to be useful.

We are honest about scope: MVP is 3 sources, Spark pipeline, basic chatbot, and dashboard. Stretch goals add more sources and features. Everything runs at zero cost on free tiers and open-source tools.

We own this work. We can explain every design decision and every data transformation. We wrote tests for every critical component, including explicit tests for LLM failure cases. The thinking is ours.

12. References

1. Hou et al. (2024). Bridging Language and Items for Retrieval and Recommendation. arXiv:2403.03952. Dataset: huggingface.co/datasets/McAuley-Lab/Amazon-Reviews-2023
2. Yelp Open Dataset. business.yelp.com/data/resources/open-dataset/
3. World Economic Forum (2021). Fake online reviews cost $152 billion annually.
4. Lewis et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.
5. Apache Spark: spark.apache.org | FastAPI: fastapi.tiangolo.com | ChromaDB: docs.trychroma.com
6. sentence-transformers: sbert.net | Ollama: ollama.com | Gemini API: ai.google.dev
7. Scrapy: scrapy.org | Airflow: airflow.apache.org | Streamlit: docs.streamlit.io

Appendix

A. Sample Prompts

```
SYSTEM: Extract product aspects and sentiment. JSON only. USER: "{review_text}"
Output: [{"aspect":"str","sentiment":float,"quote":"str"}]

SYSTEM: Answer using ONLY retrieved reviews. Cite [source, date]. Never fabricate.
RETRIEVED: {context} USER: {query}
```

B. API Schemas (Pydantic)

```
class AspectSentiment(BaseModel):     aspect: str     sentiment: float  # -1.0 to 1.0
evidence_quote: str  class ChatResponse(BaseModel):     answer: str     confidence:
float     sources_used: int     citations: List[dict]     faithfulness_score: float
```

C. Repository Structure

```
reviewpulse-ai/ ├─ docs/           # Proposal, Codelabs, diagrams ├─ poc/            #
EDA notebooks, demo scripts ├─ src/scrapers/   # eBay, iFixit, YouTube custom
pipelines ├─ src/spark/      # PySpark jobs ├─ src/llm/        # Ollama extraction,
RAG, fake detection ├─ src/api/        # FastAPI ├─ src/frontend/   # Streamlit ├─
dags/           # Airflow DAGs ├─ tests/ ├─ .github/workflows/  # CI/CD └─
requirements.txt
```

D. Diagrams Checklist

The following diagrams are included in this proposal:

1. **System Architecture (Section 5.3.2):** 7-layer vertical stack with arrows. Simple boxes, 1–2 colors.
2. **Data Flow Diagram (Section 5.3.3):** Horizontal left-to-right flow. Sources → Ingestion → Spark → LLM → Storage → API → Dashboard.
3. **RAG Pipeline (Section 5.5):** Linear: Query → Embed → Retrieve → LLM → Validate → Response.
4. **Gantt Chart (Section 6.2):** Horizontal bars for M1–M6 across 14 days.
5. **Schema Mapping (Section 5.1.1):** Already shown as a table. Optionally draw as a visual mapping diagram.